

Alemannic zkPoP v1

A redacted disclosure of a machine-checked adversarial
game-theoretic model of a king-of-the-hill network

Artifact `allemanic-zk-pop-v1`

Ledger of 906 committed claims
algebraic root 398451496810673064

Abstract

This document publishes a commitment ledger over the 906 machine-checked theorems of an adversarial game-theoretic model of running a decentralised king-of-the-hill training network, together with a single field element — the ledger root — and the protocol by which a reader can check it. The theorem statements, their proofs, the thresholds they identify and the design rules they imply are *withheld*. What the reader gets instead is (i) an exact count and shape of the development, (ii) a root that binds the publisher to it, (iii) a challenge protocol that converts a handful of opened slots into an exponential bound on how much of the ledger can be fake, and (iv) a proof that the publication itself gives nothing away.

The redaction layer is not asserted, it is formalised. In the Lean development that accompanies this artifact, `ZkPoP.Alemannic.root_admits_every_content` proves that the published ledger and root are reproduced exactly by *every* assignment of contents to the slots, so the document excludes no claim; `ZkPoP.Alemannic.ledger_root_tamper` proves that the root is nevertheless sensitive to a single leaf; `ZkPoP.Alemannic.challenge_pass_prob_le` proves the cut-and-choose bound $\binom{N-b}{k} / \binom{N}{k} \leq ((N-b)/N)^k$; and `ZkPoP.Alemannic.commit_not_binding` proves — rather than assumes away — that the algebraic commitment layer on its own does *not* bind, which is why the published leaves are SHA-256 digests. The concrete ledger’s length and root are themselves theorems, checked by the Lean kernel: `ZkPoP.Alemannic.ledger_length` and `ZkPoP.Alemannic.ledger_root`.

The document also carries an actual zero-knowledge proof. A Pedersen commitment to the digest of the entire withheld witness is published together with a Schnorr transcript proving knowledge of its opening; the verification equation, on the published numbers, is itself a Lean theorem, `ZkPoP.Alemannic.transcript_verifies`, checked through a verified square-and-multiply (`ZkPoP.Alemannic.powMod_eq`).

1 What this document is

A finished piece of work has two audiences that want incompatible things. One wants to know that the work exists, is complete, and says something; the other wants the work. This document serves the first without serving the second.

Three roles are involved.

The publisher

holds the development: 906 theorems, their statements, their proofs, and a per-slot salt for each. This is the *witness*, and it never appears here.

The reader

holds only this document. From it alone the reader can recompute both published roots, confirm that they are the roots of the published leaves, and confirm that the published leaves are compatible with literally any content — so that reading the document teaches nothing that could be copied.

The challenger

is a reader who does not take the publisher’s word for it. The challenger names k slots; the publisher must open each one — revealing its salt and its canonical record — and the reader recomputes the leaf. §6 gives the exact number of challenges a dishonest publisher survives.

Disclosed here. The number of committed claims; the number of groups they fall into and the size of each group; the 906 leaf residues; the SHA-256 Merkle root; the algebraic root; the digest of the salt seed; the digests of the prose write-ups; the commitment and verification procedures in full; the theorems proved *about* this disclosure; and one demonstration opening, chosen as the least informative slot in the ledger.

Withheld. Every theorem statement and proof; the names of the theorems and of the modules they live in; the section titles of the write-up; every numerical threshold, design rule and parameter the model identifies; and the salts. None of these can be recovered from anything printed here.

2 The subject, at the level of shape

The committed development is a game-theoretic model of an adversary running against a decentralised king-of-the-hill training competition: a network in which challengers duel a sitting king and a promotion rule decides who is crowned and paid. The model is stated in exact arithmetic and every claim about it is a theorem. What is disclosed about it is its shape.

measurement	value	checkable from this document?
committed claims (theorems)	906	yes — count the leaves
groups (one per source module)	53	yes — see Appendix A
largest group	36	yes
smallest non-empty group	9	yes
mean group size	17.1	yes
groups with no theorem	1	yes
each claim opens to a checkable Lean theorem	—	only by challenge (§6)

The last row is the honest one. Nothing printed here shows that the committed claims are *true*, or even that they are theorems; the roots show only that they were fixed before publication. Truth is established by the Lean build of the withheld development, and is transferred to the reader one opened slot at a time.

3 The commitment scheme

Leaves. Claim i has a canonical record — its module, its name and its source text, serialised deterministically — written c_i , and a salt s_i derived from a master seed. The published leaf is

$$\ell_i = \text{SHA256}(s_i \parallel c_i), \quad i = 0, \dots, 905.$$

Salting is what makes the digest uninformative: without s_i , a reader who *guesses* c_i cannot confirm the guess.

The SHA-256 root. The leaves are folded into a binary Merkle tree, duplicating the last node of an odd level, with internal nodes $\text{SHA256}(01 \parallel L \parallel R)$. This is the layer that binds, and it is computed by the generator script, outside Lean.

The algebraic root. The same leaves are reduced modulo the Mersenne prime $p = 2^{61} - 1$, giving the residues $r_i = \ell_i \bmod p$ printed in Appendix B, and folded with the compression function of the accompanying development,

$$\text{compress}(a, b) = a^2 + b^2 + ab \pmod{p},$$

$$R = \text{compress}(\dots \text{compress}(\text{compress}(0, r_0), r_1) \dots, r_{905}).$$

This is the layer that is *proved*: R is the value of a closed term, and the Lean kernel checks its evaluation in `ZkPoP.Alemannic.ledger_root`.

Published values.

quantity	value
committed claims N	906
modulus p	$2^{61} - 1 = 2305843009213693951$
SHA-256 Merkle root	c00f5e80f5a21e7fc8da71bc8f864787e844e84cc11def16e8fa09830046af2a
algebraic root R	398451496810673064
salt seed digest	f9c4a018ac7da01fd022194cc0552eb0b4b29569e4caa478a8a0097387f6ae62
source-tree digest	8b92e3ed736c890060add2650f0fa8a637110c11570fd6dc404c5e7ce6654c53
ADVERSARY.md	67ee143597d2a8c9c4c3a7818c11e1193ec8ea72769b380688743e3fe00cf138
ALEMANNIC.md	b99c7e1620e73193ebd317538ba4e2303de2d7e3bb6a3d3ff5e3574ae3005acf
ZKPOP.md	f67e8b663ef9e3658468cf092545b1b2f8409ab905e7744f3c7c06105f355230
ZKPOP-V1.md	791bb3823f31b613829e678175feee835c226d1f161f88b2d353d102ee0dfc98
README.md	4d9f5caa2c8305e2501fda3517fd99f5c916089827c74172f2d0e4182fc21145
STATUS.md	412e534a54177024bc9619c8ef2a90fbfeae3ae9f8fd5d19bf5babdb40d55403

The seed digest binds the publisher to one salt schedule: the salts are a deterministic function of the seed, so a publisher who later produced a second, different set of salts would have to present a second seed with the same digest. The document digests do the same job for the prose write-ups, which are not otherwise disclosed.

4 What is proved about this disclosure

The following are theorems of the accompanying Lean development, in the modules `Disclosure.lean`, `Sampling.lean` and `AlemannicV1.lean` under `RequestProject/ZkPoP/`. They are about the *disclosure*, not about the withheld subject, so they are stated here in full.

theorem	what it establishes
ledger_admits_every_content, root_admits_every_content	Under the hiding hypothesis, for <i>every</i> list of proposed claim contents of the right length there are salts committing them to exactly the published leaves, with exactly the published root. The publication excludes no content.
publication_indistinguishable	The two-content form: one publication, two arbitrary contents, nothing to choose between them.
commit_not_binding, not_compressInjective	The algebraic compression is symmetric, hence <i>not</i> binding: payload and salt can be exchanged. Binding comes from the SHA-256 layer, and this theorem is the reason that layer is there.
chainRoot_injective, commit_binding	Conversely, <i>if</i> a compression is injective then a root determines the ledger of a given length. The hypothesis is stated, used, and — for the algebraic layer — disproved.
opening_satisfies_gadget, gadget_gives_opening	An opening that checks against a published digest is exactly a satisfying assignment of the development's existing Merkle-path constraint system: verifying a disclosure needs no new machinery.
root_escrow	The root can be split N -of- M over carriers: any $ s + 1$ shards determine it, and the shards of the carrier set alone are consistent with every value of it.
challenge_pass_count	A publisher who cannot open the slots in a set B survives exactly $\binom{N- B }{k}$ of the $\binom{N}{k}$ challenges of size k .
challenge_always_caught	If fewer than k slots are good, <i>every</i> challenge catches the publisher.
challenge_pass_bound, challenge_pass_prob_le	$\binom{N-b}{k} / \binom{N}{k} \leq ((N-b)/N)^k$: credibility grows exponentially in the number of opened slots, while the disclosed fraction grows only linearly.
ledger_length, ledger_root, ledger_root_field	This ledger has 906 slots and its algebraic root is 398451496810673064, both checked by the kernel; the second is restated in $\mathbb{Z}/p$, where the general theory applies.
ledger_root_tamper	The root is not blind: zeroing a single leaf changes it.
alemannic_zkpop_v1	The disclosure in one statement: length, root, redaction and audit bound together.

The headline statement reads, verbatim:

```

theorem alemannic_zkpop_v1 (h : SaltCovers (ZMod pAle)) :
  leavesNat.length = 906 ∧
  natChainRoot leavesNat = 398451496810673064 ∧
  chainRoot leaves = (398451496810673064 : ZMod pAle) ∧
  (∀ contents : List (ZMod pAle), contents.length = 906 →
    ∃ salts : List (ZMod pAle), salts.length = 906 ∧
      List.zipWith commit contents salts = leaves ∧
      chainRoot (List.zipWith commit contents salts)
        = (398451496810673064 : ZMod pAle)) ∧

```

$$(((906 - 91 : \mathbb{N})).\text{choose } 40 : \mathbb{Q}) / ((906 : \mathbb{N}).\text{choose } 40 : \mathbb{Q}) \leq 1 / 66$$

The hypothesis `SaltCovers` is the hiding property of the commitment — that varying the salt reaches every digest — carried explicitly rather than assumed of the concrete compression. Every other clause is unconditional.

5 How to verify this document

1. **Recompute the algebraic root.** Take the 906 residues of Appendix B in order, start from 0, and fold with $\text{compress}(a, b) = a^2 + b^2 + ab \bmod (2^{61} - 1)$. The result must be 398451496810673064. This needs nothing but integer arithmetic, and it is the same computation the Lean kernel performs.
2. **Recompute the SHA-256 root** from the leaf digests, and check that reducing each of them modulo p gives the residues of Appendix B. This ties the binding layer to the proved layer.
3. **Check the demonstration opening** below.
4. **Challenge.** Name slots. Each answer is a salt and a canonical record whose SHA-256 must equal the published leaf.

The demonstration opening. Slot 378, chosen mechanically as the shortest record in the ledger and therefore the least informative one the publisher could open. Its canonical record is

```
{"module": "RequestProject/Adversary/Fork.lean", "name": "mass_empty", "source": "lemma mass_empty : F.mass \u2205 = 0 := by simp [mass]"}
```

with salt `f2b2aab36ad815804046c561b170520e8049cd6901beaf921c1d6644faee884c`, giving leaf `493a49b0e1e0a70e36e5ddd1eec90a76a0c3fb4cded9077e9d0e919d5a0a1d0`, which is the published digest of slot 378 and folds up its 10-step authentication path to the published SHA-256 root. One slot of 906 has been disclosed; 905 remain closed.

6 The challenge protocol

Suppose the publisher can open only some of the slots, and let b be the number it cannot open — slots committing to nothing, or to something that is not a checkable theorem. A challenge of size k is a uniformly chosen set of k slots, and the publisher survives exactly the challenges disjoint from the bad set. The proportion it survives is $\binom{N-b}{k} / \binom{N}{k}$, at most $((N-b)/N)^k$.

With $N = 906$ and $b = 91$ — a tenth of the ledger fake — the numbers are:

challenge size k	exact survival proportion	bound $((N-b)/N)^k$	ledger disclosed
10	3.450e-01	3.470e-01	1.1 %
20	1.176e-01	1.204e-01	2.2 %
40	1.313e-02	1.449e-02	4.4 %
80	1.387e-04	2.101e-04	8.8 %
160	7.340e-09	4.413e-08	17.7 %

The row $k = 40$ is the one carried in Lean, as `ZkPoP.Alemannic.audit_v1_bound`: a publisher with a tenth of the ledger fake survives a 40-slot challenge with proportion below $1/66$, while the challenge itself discloses 40 of 906 claims. The asymmetry is the point — confidence is exponential in k , disclosure is linear.

7 A zero-knowledge proof of possession

The ledger binds the publisher to a set of claims; it does not, by itself, demonstrate that the publisher holds anything. This section does, and it does so in zero knowledge.

Let W be the SHA-256 digest of the entire withheld witness — every salt and every canonical record, concatenated in slot order — reduced modulo q . In the prime-order subgroup of $(\mathbb{Z}/p)^*$ generated by g , with a second generator h obtained by hashing a fixed string (so that nobody knows $\log_g h$), the publisher publishes the Pedersen commitment

$$C = g^W h^r \bmod p,$$

and then the three-move proof of knowledge of (W, r) : announcement $t = g^a h^b$, challenge c , responses $z = a + cW$ and $w = b + cr$ modulo q . The verifier checks

$$g^z h^w = t \cdot C^c \pmod{p}.$$

All nine numbers are in Appendix C, and the check needs nothing but modular exponentiation. What this buys, and from which theorem:

- **Completeness** — an honest publisher always passes (`ZkPoP.Schnorr.completeness`).
- **Soundness, as extraction** — two accepting transcripts on one announcement with different challenges *yield* the opening (W, r) , by an explicit extractor (`ZkPoP.Schnorr.extract_sound`, `ZkPoP.Schnorr.special_soundness`). A publisher who could answer two challenges but knew no opening would therefore have produced one.
- **Zero knowledge** — for a fixed challenge, the accepting transcripts are *exactly* the ones a witness-free simulator produces (`ZkPoP.Schnorr.hvzk`, `ZkPoP.Schnorr.simulate_accepts`). The transcript in Appendix C therefore carries no information about W .
- **This transcript** — the verification equation on these numbers is `ZkPoP.Alemannic.transcript_verifies`, evaluated by the Lean kernel through the verified square-and-multiply of `ZkPoP.Alemannic.powMod_eq`. The equation is stated with honest exponentiation, $g^z h^w \bmod p$, and only the *evaluation* goes through the fast form.
- **Hiding of C** — `ZkPoP.Pedersen.perfect_hiding`: every candidate W opens C for a suitable randomiser, so C excludes nothing either.
- **Binding of C** — `ZkPoP.Pedersen.binding_extract`: two openings of one commitment with different messages *exhibit* $\log_h g$. Binding is a reduction, not an assumption.

8 Non-claims

Read this section before reading anything above as a security guarantee.

- **This is not a succinct argument system.** There is no polynomial commitment scheme and no proof-size or verifier-time bound. Verifying the root costs a fold over all 906 leaves, and answering a challenge costs one opening per slot named.
- **No cryptographic hardness assumption is discharged.** Hiding of the algebraic layer is a hypothesis, named and carried in every statement that uses it. Hiding of the published layer is the usual salted-hash argument about SHA-256 and is not proved here.
- **The algebraic layer does not bind**, and this is a theorem, not an oversight: the compression is symmetric. Binding of the published ledger rests entirely on SHA-256, which is computed by the generator script and is outside the Lean kernel.

- **Nothing here proves the withheld claims are true.** The roots prove only that the ledger was fixed before publication. Truth reaches the reader by opening, and the cut-and-choose bound of §6 is how far a bounded number of openings gets.
- **The transcript is non-interactive by the Fiat–Shamir heuristic,** with the challenge taken as $\text{SHA256}(p|q|g|h|C|t) \bmod q$. That step is *not* formalised: the Lean theorems are about the interactive protocol, for a challenge the verifier supplies. A reader who wants the formalised guarantee should send a challenge of their own.
- **The proof of knowledge proves knowledge of *an* opening of C ,** not that the opening is the witness digest of the ledger. Linking the two means revealing W , or opening slots; it is not established by the transcript alone.
- **Discrete-logarithm hardness is not discharged.** The soundness of the transcript against a cheating publisher, and the binding of C , are reductions to it; the group is a 2048-bit prime field with a 512-bit prime order subgroup, and its parameters are generated by the script and re-checked by the verifier, not certified here.
- **No trusted timestamp is used.** A root binds the publisher only against a record of when it was published; anchoring this root externally is the publisher’s job and is not part of the artifact.
- **The shape data is trusted.** That the 53 groups correspond to source modules, and that each slot holds a theorem rather than a definition, are claims about the withheld material. They are checkable only by challenge.

Reproducibility

The artifact is regenerated by `scripts/zkpop_alemannic.py` and checked, from the published data alone, by `scripts/zkpop_verify.py`; this document is emitted by `scripts/zkpop_report.py` from `zkpop/public.json`, so no number in it is transcribed by hand. The Lean side is, under `RequestProject/ZkPoP/`, `Disclosure.lean`, `Sampling.lean`, `AlemannicLeaves.lean`, `AlemannicV1.lean`, `ModPow.lean` and `AlemannicTranscript.lean`, checked by the development’s build-time audit, which admits no unproved statement and no axiom beyond `propext`, `Classical.choice` and `Quot.sound`.

A Group sizes

One group per source module of the committed development, in the ledger’s slot order; the module names are withheld. The sizes sum to 906.

id	n	id	n	id	n	id	n	id	n
G01	21	G02	24	G03	10	G04	25	G05	13
G06	11	G07	9	G08	12	G09	13	G10	18
G11	13	G12	25	G13	9	G14	25	G15	10
G16	24	G17	18	G18	18	G19	32	G20	16
G21	14	G22	0	G23	15	G24	19	G25	9
G26	10	G27	14	G28	13	G29	16	G30	16
G31	15	G32	36	G33	14	G34	13	G35	25
G36	11	G37	16	G38	21	G39	20	G40	14
G41	10	G42	17	G43	13	G44	11	G45	17
G46	23	G47	21	G48	16	G49	22	G50	28
G51	23	G52	30	G53	18				

B The published leaf residues

The 906 residues $r_i = \ell_i \bmod (2^{61} - 1)$, in slot order, prefixed by their slot index. Folding them with compress from 0 yields the published algebraic root **398451496810673064**.

0: 390596343188301325	1: 1184053191673673270	2: 340635574613167067
3: 202562657393452890	4: 443673185874325519	5: 1490157747096130708
6: 1962525924425329657	7: 424988682057194989	8: 49762172986207912
9: 510704654817405151	10: 2257455718306467201	11: 85663639653618277
12: 540373433503148672	13: 1540426587056675321	14: 760012082950836078
15: 767919341762968428	16: 1154388267254336125	17: 1078897562770267640
18: 797296439616500722	19: 109817919082055877	20: 90704968916688735
21: 1708178876053484459	22: 893585205197033831	23: 669933170354670468
24: 1268251274189422562	25: 1243776379577282553	26: 638687457091248378
27: 1117918882994358264	28: 2060438253756117699	29: 841257686714158155
30: 2018244828322512628	31: 1125385665070185831	32: 489701619645649096
33: 947459044826875256	34: 2200083522754048459	35: 2042549246594884484
36: 119064673161508602	37: 1034158280487316154	38: 868166025547204258
39: 459874513746106862	40: 1880865367528251616	41: 63610760449181037
42: 1569564456266631935	43: 1482989141562494690	44: 1400355612893943870
45: 1177671204562778944	46: 1095533250126016035	47: 882066299948860055
48: 2035501444652380806	49: 2230244743562388997	50: 1600289445253331988
51: 2001028843832871328	52: 415015544361886717	53: 169755746252623678
54: 340794061801112637	55: 126826495206217931	56: 1230351063457015278
57: 1926955043072670413	58: 989947502682650579	59: 1315797163561221613
60: 1007979682307581375	61: 492037642784945233	62: 1902436025041179954
63: 2123144706326311266	64: 300605539375843993	65: 1341219671061281282
66: 2060440171347294689	67: 747705715401790026	68: 177023063619709840
69: 8520438394712704	70: 1649556361891760298	71: 598311394511532456
72: 345102263230405922	73: 581942078468177395	74: 860011626320155971
75: 2080030057765193700	76: 1001550771062573275	77: 2086771377994775699
78: 969993735337508460	79: 177174978865963736	80: 318077316048297675
81: 862975902811538873	82: 1095621227852320635	83: 32483329694640955
84: 589938092219505066	85: 1163326052830992190	86: 2286637050642033522
87: 2148034928680134952	88: 1652041830094322295	89: 1751375335378536111
90: 294423897684504083	91: 1927363273881142509	92: 728470873114249184
93: 941366844885050940	94: 2010319344160511110	95: 1175015712848640683
96: 429685632619506390	97: 925477370881468845	98: 346636775403829328
99: 61108969852554338	100: 1615881463227620587	101: 1706880216255802132
102: 719856847172006380	103: 2108155550261884504	104: 1197095514717146153
105: 597620921563973395	106: 963379507102890148	107: 2173805494295118971
108: 883738322200231505	109: 1921557247896768746	110: 403460912738952081
111: 2157000627003845895	112: 120790229051037051	113: 934667889957124539
114: 769480219794840452	115: 2140539356247931452	116: 472707107396288991
117: 997726571097416285	118: 138350075558403224	119: 1352746241412102939
120: 432925649550049818	121: 1762432157584913727	122: 1425240078880415167
123: 379431194262952514	124: 72938595601764928	125: 699259661214916729
126: 887831364596544098	127: 754999768225258097	128: 151302028784254926
129: 590103136407539336	130: 665849529441739863	131: 692785821555887366
132: 1041841415715135664	133: 1584807075241822982	134: 1200891169672168216
135: 1831486383478088936	136: 1242663035961975020	137: 2145612404675167279
138: 2256465127238033587	139: 252208599669795466	140: 2026935188390121894
141: 1052935798906658592	142: 1355194928936150216	143: 1979030021298481400
144: 1474888063362419040	145: 2173240135313805597	146: 1795222220642736764
147: 1743847097394517025	148: 332544728257377365	149: 253228177168030932
150: 1756807501961069333	151: 1230669897287189468	152: 518389084498925797
153: 2057594427606870582	154: 66055473537522967	155: 1834847711112583457
156: 1528970877297742026	157: 1159584615064884242	158: 711739012060147614
159: 779602935315071185	160: 538823949707499798	161: 1707510405891091852
162: 4892072220599193	163: 285576389521889026	164: 1522041337328441238
165: 1163995970521510238	166: 1539728740228084885	167: 604921479324599560
168: 1049665642654714436	169: 2125602373753225723	170: 1537148764286026024
171: 2247749466707485166	172: 52903508682970901	173: 1991762775143394454
174: 1055441962962475600	175: 1251617549635079527	176: 183689671714632125
177: 1896466466670705267	178: 1453329375984408748	179: 2157108636038527858
180: 953768779428639992	181: 24377500995081854	182: 1900697290935972830
183: 245219314052631723	184: 1435487086920211931	185: 1500501830165771590
186: 1569662355714538509	187: 633100061605892248	188: 1930165213717910747
189: 1210596381624315900	190: 200993570493523127	191: 279946803383358688
192: 2121421464156637623	193: 1159128047860507993	194: 1367212984242260320
195: 239915099211182690	196: 321325999794399607	197: 1593447850626172481

198: 1399609212114395775	199: 1468709298802056139	200: 466924433745792166
201: 1589728799573907695	202: 514912268710441146	203: 84779240903475461
204: 696620825498528246	205: 225916328630496661	206: 104926514368506365
207: 1990743143244905110	208: 290493387366026771	209: 430459555167311065
210: 2205900426282148273	211: 1137321766177985596	212: 1817575131433025946
213: 2036453053341064385	214: 521455846626097864	215: 559488299760336509
216: 1745118564666351375	217: 568943596295541545	218: 1642077904141100009
219: 489330887885639719	220: 706021306811015646	221: 2234483009635848614
222: 1839389979530930021	223: 1256102512319706981	224: 1457706569663511705
225: 373912770943981773	226: 730289020492621228	227: 410988702913435356
228: 650721928559334977	229: 1831939730525914371	230: 1918773997919744170
231: 1746895571876010563	232: 1329144067624869105	233: 144625168218030675
234: 1728236817449504675	235: 1778568162646437025	236: 1157347969834948611
237: 1472349960039117111	238: 1293499330276830546	239: 1271096511917188384
240: 1709452005227475465	241: 1962254875772542584	242: 1853694691434184217
243: 1843813593187270982	244: 1706740709499172985	245: 1173020977576982523
246: 1336338295334174040	247: 1520575569945627437	248: 562708909529501823
249: 357350875076377436	250: 708884740431935115	251: 396314346867424249
252: 1584266629350522812	253: 1455829422887695837	254: 89445227478310859
255: 27894544107081163	256: 1346477424316939349	257: 737834434294057736
258: 203769152623514677	259: 376725612232809338	260: 2090596482306479158
261: 748338628626865828	262: 856625404119815723	263: 1733774013499495127
264: 1012018824105676737	265: 933527743507504692	266: 1612421672002771554
267: 790036805013754316	268: 168015162794392154	269: 1179130027345928700
270: 724608879172287178	271: 1248544378593791431	272: 827790699159559463
273: 646377078944267283	274: 2202873595177692106	275: 1948259210272939181
276: 657065847998245867	277: 539031270915242742	278: 1376097449012616330
279: 1787673845917068093	280: 1905630581839084981	281: 527017399717868058
282: 661462142855657338	283: 1197876390546305055	284: 1343453072932944302
285: 2259057819721339222	286: 921179533248547240	287: 433924429569670814
288: 1897698581568022498	289: 650463307882119129	290: 2304245954960718684
291: 1621006028567289175	292: 848048460180178109	293: 1905069222613307012
294: 899727241664562781	295: 65381252735552861	296: 605852318132912801
297: 1239144013944270942	298: 2222587770728013423	299: 2245673449585680881
300: 1701652037916328810	301: 1445206709053520941	302: 632195891060884837
303: 804609818893524066	304: 1608479724117955663	305: 1182673224084880884
306: 86431931726180972	307: 838731004023931911	308: 104910416915743884
309: 56607777495586206	310: 2274348133975562907	311: 629623542345916267
312: 124638568087553460	313: 250362308339821382	314: 1784042333948761775
315: 533197978452874823	316: 1756811980080831108	317: 683729292078184321
318: 2012908921184732114	319: 1207398373828599861	320: 1240032169917168738
321: 814233345382267892	322: 23776068398843781	323: 2217457448863100610
324: 722803625997403932	325: 1858056273057229849	326: 1089119911791872628
327: 1084986937109922868	328: 2241553211724111738	329: 2280456105701949749
330: 733453093264344871	331: 1873872692384414308	332: 1693151577943548345
333: 369129888529907414	334: 776109111615424164	335: 403434332015470161
336: 774531062481402985	337: 1524995625143903954	338: 1190131270353685609
339: 182040105188301948	340: 1502118529483477169	341: 1074525593372964704
342: 378457029567814825	343: 812040005440871017	344: 2220394985343005918
345: 1668399072278911354	346: 389355417876051923	347: 777809440750833618
348: 1213221210824523517	349: 184805304635726786	350: 1665228859942008101
351: 810288256390289725	352: 1213229486952286929	353: 1955446587095906593
354: 113892780544303131	355: 1504450777337023353	356: 464073821652752539
357: 1025630816442586904	358: 1845129291737848767	359: 950855905133506532
360: 1090664247583318007	361: 365734701239400384	362: 1557254711115037575
363: 2203212709761674763	364: 2079237825464956483	365: 149078889259090966
366: 2181127574714359086	367: 1395785378108168211	368: 184878805723965475
369: 2302693981123671851	370: 1244246052346790450	371: 759071277633055894
372: 1990264513934505607	373: 2203486026461305382	374: 894598014946825918
375: 1812396738503739381	376: 717447246148105683	377: 543663068544197005
378: 593841607728394347	379: 2243661221569531045	380: 698998699931046209
381: 1949031599678918934	382: 586710581064118105	383: 1488625355447649380
384: 1488853349070537606	385: 2300136879332162124	386: 1046134661243947470
387: 869771451850378234	388: 1012478893774720679	389: 130751801572624724
390: 712404218078734050	391: 394484550557700057	392: 541929364206670720
393: 469307756333678873	394: 1570337888216438716	395: 1902279826924929823
396: 274471065659902280	397: 2112713528999187581	398: 1266492269069691511
399: 771437048567062507	400: 1596121187831546970	401: 1181364676698566408
402: 610962841732097670	403: 4376127506875220	404: 2003220783622192373
405: 285019264318040263	406: 1855564319074774223	407: 1898495036229859697
408: 1586905964865007796	409: 2129557442364073961	410: 1431680413324043611
411: 1684276890598554910	412: 1576108566872551263	413: 1594615484468545535
414: 169351840168617707	415: 1392820243967428139	416: 2217775986574553193

417: 1195322641271867874	418: 1516292573332039574	419: 462278478574031544
420: 2265434713185160104	421: 1306646406396610870	422: 2201514144701438562
423: 1969220317143862696	424: 1572461279350272830	425: 213389406873142933
426: 2043854303575593527	427: 712245134479101733	428: 21507320672016376
429: 534710589988919807	430: 1239099364026697399	431: 1016922393767910899
432: 2055093331399363608	433: 1876985117315153246	434: 638455482963758328
435: 1717502577569199840	436: 1505324416120420597	437: 1945387030871219333
438: 974980425846175720	439: 1621976887621754436	440: 29938001551512160
441: 2209467427302884464	442: 1147630397545366950	443: 245536445458241915
444: 1482634498260828997	445: 861088497525858630	446: 21629074443306452
447: 756987695992528925	448: 2208954174738864368	449: 798986963827627530
450: 1288554676774958487	451: 1611698270432960549	452: 1084344395394408815
453: 1865239298497048536	454: 1969960867397866111	455: 915489462259538326
456: 1017542509774808391	457: 1675436517921483626	458: 2013670435654411632
459: 1100771603246755500	460: 53747299833963722	461: 236892421788998041
462: 1486111503552756646	463: 499581247186116447	464: 1451954132540624321
465: 2104326974579688428	466: 1810212902529517613	467: 1892398303845377657
468: 161561438208074219	469: 374416340925782735	470: 1424401385969174513
471: 2283286448998996953	472: 2197613393707397265	473: 1677877059252711734
474: 1076104108454273161	475: 2225055129195445594	476: 2209114949269359726
477: 1508432689560855483	478: 785434140630108266	479: 378701178335051621
480: 256722908205371822	481: 746559968727608788	482: 526520962352742931
483: 1045854819029551872	484: 389754825207425832	485: 185539000898944973
486: 1397696014069484780	487: 946229540143767640	488: 351068134722505785
489: 2291565867565733356	490: 2035374845373733347	491: 1792086784142151754
492: 979536246156500646	493: 1043049456771496802	494: 838483743274167723
495: 1024744209282561187	496: 793423349837508229	497: 1844279919299947866
498: 61632783870147044	499: 942764516545732779	500: 1994190493415739746
501: 496074978553282783	502: 975250206186975618	503: 763259980987466546
504: 1585548496125367421	505: 2295953426270233993	506: 732442217809256971
507: 1529602317051913872	508: 263389259052482981	509: 787625379738934489
510: 1207457740824534517	511: 2283944829157820827	512: 30808065100070473
513: 1150519338586717517	514: 884390649316012894	515: 1942877000487687582
516: 958262052376952303	517: 810639491565086953	518: 1202772921418845288
519: 1567398584271232706	520: 807462052551357353	521: 197228418636930107
522: 128145632761937323	523: 727108067493661416	524: 531061694869573846
525: 1832591039897785518	526: 172998753315632902	527: 1284276566542784930
528: 460045578308732956	529: 2045894166640176693	530: 1671676805333188747
531: 1876906695344166007	532: 85067156414510420	533: 1363205977011462661
534: 372379883976732056	535: 2201405005481589676	536: 1743231745186127637
537: 1610309569952643156	538: 769349810036410050	539: 1261402176137031480
540: 501144279326621620	541: 692628567954922779	542: 351649044578716309
543: 891675565725122630	544: 1991809246923287037	545: 423143711122820199
546: 1505679740385748602	547: 1909445623592268025	548: 1135528098611074332
549: 565074578135701951	550: 1827028725265525614	551: 2103737821122398312
552: 931460667649437116	553: 178164687983372588	554: 1061515187860502569
555: 2107113900611628282	556: 2062851993800284956	557: 2071558518128793774
558: 1814910484542224540	559: 1921724806857300997	560: 2257924509590150633
561: 316630795736502864	562: 2186774445302287144	563: 327632154226727835
564: 1420273647686560675	565: 1848001366719794597	566: 977529586222456816
567: 457026884994709950	568: 327400819475929321	569: 1707195599913589710
570: 1578766566282363834	571: 1490578516713541206	572: 1248153990023787543
573: 221138659222570021	574: 811208992101832143	575: 1957748494128064209
576: 449431399500514524	577: 1959178593554765913	578: 1288991395977245702
579: 1899706727523664330	580: 1736946490179125215	581: 569368706580889419
582: 1714334136895854394	583: 1025718590259197362	584: 1561880955447753122
585: 97586921696292155	586: 2043483877458764383	587: 896303039931396170
588: 449379513257115905	589: 45282135037945058	590: 1044967177703808755
591: 775667121507646044	592: 2127370137430065018	593: 1861802332828765002
594: 138025436510689439	595: 1863497759562311311	596: 1307063441424636414
597: 530800960678947844	598: 491294963661026302	599: 576373036895875085
600: 1980823584884434760	601: 93715085545549972	602: 571043483222198092
603: 987091081730601895	604: 1190319265252788575	605: 1366995913083031150
606: 505920894182173568	607: 465771801434401904	608: 1890222736495499568
609: 732866533573063876	610: 1153591927773896684	611: 1804536439669048347
612: 387752940793430751	613: 462056286560492611	614: 937091713568029897
615: 1931782314875381799	616: 265437834401453810	617: 1369507412519994662
618: 1305264361556058957	619: 2177047571342911940	620: 1704711485852362580
621: 2198845733367751461	622: 2117495874483702610	623: 487205717194334531
624: 368238037010754255	625: 1920222934804632857	626: 894319631826732577
627: 522200722088682154	628: 1506228215971284121	629: 744587831831689966
630: 2301884883329144412	631: 238090250758177194	632: 2293599595845313995
633: 1896215772981745271	634: 690972641033126734	635: 1298948062197960157

636: 987835065847537825	637: 257357382659397443	638: 1460826994042814489
639: 2031180558750115460	640: 2021859738703378315	641: 1328510820709640686
642: 223752387679463193	643: 525948900456352613	644: 941540674944591391
645: 1569313179408629610	646: 236175634408477904	647: 1418263458433527877
648: 1073288284314445991	649: 748648138143238637	650: 1808789807205861946
651: 1583226538744680128	652: 677075895391404054	653: 989653675120074745
654: 1912371999277447761	655: 675752878575074609	656: 1976455614688937555
657: 2207434146237058774	658: 2145683517103661388	659: 2301591979253182687
660: 449896906191532714	661: 1467618459022819394	662: 492885335515972016
663: 2288205449738771985	664: 1419106243520790773	665: 1660750855854701737
666: 2189028552011295844	667: 1551423635526997767	668: 544233910776247242
669: 69346852587253885	670: 1142526099408778645	671: 833438738604192640
672: 1176039723590406612	673: 2055618736093107360	674: 878278727808996125
675: 328382021137116844	676: 395226603816958791	677: 890628917158996872
678: 2162973207305262518	679: 1084494654410529264	680: 1098680829364257260
681: 357128835662014722	682: 1040679959800371792	683: 1437942269701855738
684: 1057805774172503547	685: 671667423206485428	686: 736552584815296311
687: 908091976361872224	688: 20772133598420923	689: 750218344742949420
690: 820583667355065471	691: 515743380252106015	692: 2097158691063457261
693: 1941573107563917842	694: 1439383191848586491	695: 1038788955375112448
696: 164366722835555504	697: 1709179734323684491	698: 1740284791224915834
699: 366926826076858817	700: 1705172312399150200	701: 1408272514748949365
702: 2252007368981762662	703: 634144843658609529	704: 359027720372634322
705: 1343573106170197936	706: 1058293458538549314	707: 1336620483990756514
708: 318349156584557480	709: 675561649668028546	710: 715645621011483830
711: 2096114888739727096	712: 1771827765435686294	713: 76219231155042415
714: 714413603358113329	715: 2058027569015607364	716: 677462301203159110
717: 85200286641706562	718: 2241617654458464058	719: 1946431867330316386
720: 1558743298705894883	721: 2278782796128628217	722: 764063893645153167
723: 2111559492545149677	724: 1516951196088818178	725: 1694959421376135661
726: 660179784715359526	727: 1124326085113876668	728: 407986506101659647
729: 2120272152227973527	730: 1343214651808585318	731: 904903548102398628
732: 959084538434634299	733: 1230535105176197753	734: 2205366904500725532
735: 1647401731837179662	736: 919897048519125980	737: 1853367660344101536
738: 1770220833229194433	739: 1037686723529985971	740: 858330828360756986
741: 1455035534182620398	742: 2071219252579788606	743: 2056935561888647350
744: 1707204394842724315	745: 1617322822482938351	746: 621346284222717615
747: 632104839770234024	748: 2152565991967090968	749: 2130067549402117639
750: 410869485816515632	751: 587458575464111473	752: 2093955736172506346
753: 872848437759803459	754: 721514722769243461	755: 1922021497693418332
756: 1375209936232030768	757: 186763978403824435	758: 1992869343375939710
759: 1419321540381307108	760: 1480894779801890515	761: 2040283876759919949
762: 1788731511719889859	763: 441296042859672666	764: 1919775082882242814
765: 192186843871177297	766: 2155215340553650782	767: 476040267884464029
768: 653697233812559803	769: 144730615676376389	770: 163933692764225839
771: 1797024213307127966	772: 1959792347093841366	773: 1861105818900787180
774: 472887159663881457	775: 1954299114849335537	776: 1963299892311421249
777: 1998080722735297166	778: 1288868911480728852	779: 1750397533582193963
780: 798297448904750819	781: 655991551231217582	782: 1309728539059854712
783: 1111548966434900596	784: 380654392105982269	785: 633352514043385425
786: 810049414583477447	787: 1575527778269255723	788: 688605702327617015
789: 1658358523569757208	790: 2026595911257804572	791: 1578152553465209176
792: 1339218584188752297	793: 133196975268650504	794: 1177041448326257337
795: 1466991554873261855	796: 622590360542230266	797: 1054370154821792272
798: 353001967987944326	799: 279208626396055285	800: 1864149150370837280
801: 693506925526680502	802: 1474085613469984705	803: 247480201194410218
804: 2028879062760516509	805: 281547769981272239	806: 1844461047070926284
807: 2041378014703895288	808: 1858817972442957461	809: 894919033890675228
810: 873390738742066620	811: 477043085475948297	812: 2190603398316694576
813: 1018658163517664699	814: 2281921111738695646	815: 1458090950554323827
816: 737432334141957944	817: 1090347341145448265	818: 1702009507255581063
819: 549464529551926035	820: 1100958098231163888	821: 160275459846232427
822: 1503837912648622229	823: 1135643848410163544	824: 1125457375341989371
825: 1944771388251519025	826: 1128177167446781333	827: 1686802511793404324
828: 590226492637899115	829: 673621783873489485	830: 1279999316449779429
831: 1290392655610994313	832: 470513610604517658	833: 2267252533103339463
834: 1025108072995403359	835: 405206866482962921	836: 216283890637253554
837: 1068881537116633650	838: 531055128272579733	839: 1566841967298000418
840: 2073834648281338266	841: 706871716541292843	842: 1039825993415953685
843: 626702561082592255	844: 379097845581534406	845: 1928718189129175260
846: 894831905752029841	847: 1383113431818130251	848: 1652564154260501181
849: 356154059832616090	850: 1632791397353373956	851: 1022445579234174546
852: 1710586245454542905	853: 236817896219779829	854: 213381382741242759

855: 1035790412404208939	856: 2122355786013825711	857: 1419857827287864574
858: 545946007089748661	859: 1977008959896617508	860: 1342925572617030569
861: 1431460380144717669	862: 1964039116898869333	863: 2304706544539318805
864: 1668229518069280123	865: 1213989137005185232	866: 1004455431301690672
867: 1364718914007960443	868: 834325925378083638	869: 2094277666780692457
870: 798368594125776422	871: 817828627899537439	872: 300520591031726434
873: 2215972130402377530	874: 2082020771597313489	875: 1875037775233869112
876: 1592582374704109231	877: 1050273981685401630	878: 1341150812063225747
879: 70541834585265926	880: 889121007211649458	881: 1618977849309256033
882: 1482675644782051595	883: 1199323596805861641	884: 626051830389492619
885: 2226788736640132473	886: 1266223645593278357	887: 1695216459788953731
888: 862919403235977576	889: 1475368495528649480	890: 2175074395334726128
891: 1997552049303731780	892: 1876012290558538820	893: 1410190901788320208
894: 2078312414868562241	895: 375054741784218245	896: 846312103371154629
897: 1663962656513734122	898: 1938465003755688054	899: 2029160989564305579
900: 2012833637833287535	901: 32338850068089382	902: 1675565667450865437
903: 1187940768910450692	904: 2246201845140782611	905: 1179584360502361100

C The proof group and the transcript

The nine public numbers of §7, in hexadecimal. The verification equation is $g^z h^w \equiv t C^c \pmod{p}$; the challenge is $c = \text{SHA256}(p|q|g|h|C|t) \bmod q$ with the six numbers written in decimal and separated by |; and h is $\text{SHA256}(\text{alemannic-zk-pop-v1/generator-h})^{(p-1)/q} \bmod p$. The digest of the withheld witness is 4b08e2bd7e9f2c2503fbaee949acd1b0a0ed990b03ef682bf321ca78f40121, committed to by C and not revealed.

modulus p (2048 bits)

b21deb04eca367646deddcd882452ce8d56ad60be1b1a2cbe426f31b8190e3e30ea0f153fb259e55d5c8b0e679d67155
e61adc0d4cfd0a9e240127787eae33546fbf955a971d475e01b454f0b92a22282bfde7ed4e9a5f3824cb1d5cf7686fa0
22efefe2368be41f60b3ff96df44fecf40a68ae8dd93823cf3dfc58b0bd8b94400395bcd2a4b0a834fc679dde457e36
943fe55c436db45ec39fb86057540d849515aac4b008b9bd377a4940769c07f354a514fbf9e449f2037a2189cfbcafe9
f2dbe40b7887ef7d3fd1df7d452c67a353f2702895bb7504eede01ce4e06d633e914f8eec41169d9d415edd6c4dac3a8
ae9ca58d9b625b17f036076c1f5dc4c5

subgroup order q (512 bits)

d19e11019d0eff20b812e535f8470ed8e3b9969966cea5421d32e85e795276546cf5e7872fbacd5fbbacdae7439cd806
94b9ba9515c5b3bed775ecd09c518cd3

generator g

3a97e9aff396a551806bbacaca51143c33f666049e4a5e66febada7b99bf551a87f2c93a85039b98e8fae98a42c85364
bf68d74b010c018b974a9d8676d48e24ff6ad8fb2b634f5546970c655a923cc9c96bd09a6f2d9d6a3e6e8f4e7fa1e50b
a40ea6cf6bcb4a0c0ce5df1bf516eb16b87dc38ac26edad50ee71cd1118c31d6ec525186b2adc7a4694e75201606de5
ae5fdb886304328e80a5993f9875eeb37b565f02d2905a4531be7a5109effb59fc37072f8ba2f2adfef4ce8c4c8d1716
b9a33a24b96d20a68fec3694cb9b035589a0b2f684c2246b530023047ef532dcd875fbc3bdb3a9f88bb8d837d3a6e725
9ac421343c88247545c43c3663406e35

generator h

0985bd088286b82e69881794c8d172fdbafa5f148f699eaa37db90c6d06dd070b033fea39d7f8290f025048266aff479
50e2e187a3c08d405d538b5a58a981b282edf45ff80616fb1fe1a86e3e31872cb0b0351859805ee45554683e88e21b52
24ffbb94daba45d906a41767d893398e1822da83fad3b04d94f9f7c078c1325a36aa053c892bd26ab4cac529075b8090
9a6f9fbb25191bf8981e697cebd0272f180721b7082f6b628d3ef5daa72fcce9cb85782f5d12b564c07b8c12dba7dfcc
594247f663a8820e31b4eeb7f5647a74d4cc3a142866ffddf9fb54c32c421103ea67d4bacb5c5ef796de8f6bade77391
722e22fb99189ad2b1ecaaa6e4b9a6ef

commitment C

886c0a9aafb305ae555696241b420d3d9ed902e42306fdc43475e89061e5940fcaefdec03611b6c2167e204e59399e54
8ac55968966b3115c0b1dac13fc15cbcb0e2597e5debf67dddac8f0fbafbffa0c9574f8831efa664d0f769884fa143e
ba0c8ee7a21194466cf677fc5bb9599f3fa0c782433df90a0f4f01f2baa08e502222e63d3fe454960bd6c619246fd6e
c9be1c08f12823428051ccc4ce28a5a213bd1e46acb5456b594edf2b60b1b1da8b3871444afd66f68bec9ddcf44167a3
e2354a4aa220f93e0b0b20abbbfb587ffaf4703c3cb633bad810dff02083869c515fa4a1ec74d55e3bff29418759d3012
60a9888210f57a29d10f2065b1313168

announcement t

2df5801dc46ed9d853d17f41ccf64b45e29117196d533797576536488f31d6f3e9a00fa58c99c7b207f7d8700ed769a8
636e21edf648297380ebdded170db12f798081e87db8b5c42f7ecbb7b2f1e8e071d37143da80372bcb02593f1e36f525
55b010b5a9a9c06a000685967faeea3406b947f897b1e49a48f20c5a416430b176b9e3a56e23c8475b5bdf583cc34f77
d8b358892e819cf9fd7dbafcd2c5967fc7dc7d19e4fc117a70a29a39086b47257d2b0f05547094fe0597937e9f835bef
5dc0818732c856134388d69d5bef745bda7dbea582500fc2eeeebd249b1822f78209ab5b55128512fd97f2db81b7aeae
60f1a3413beb80fc2978867c29db0b9c

challenge c

400e434034d1092eb6bd3da1699bc7244341bb750874d96624a2c70780ca3178

response z

a9d5885ace8fbc39a4b9fae0fe422af266fd3e07cf26369241ce76e9ca96030f67c846e747e0fb4ebbed1200698c84f0
9e99d93797f378bac60f392a40dfbdde

response w

382fa9aa422233b4f2bb029f3b5f7f889a64dd2f54e12d012bea3b2a59e817134cd573e0d4da4b7298d1e509e696ecd6
cbbd82faaed3a0bb967be435a6d1342a